TDT4259 – Group Project

Day-Ahead Hourly Electricity Consumption Forecasting

for a Nordic Electricity Retailer

Aneo case

Nils Henrik Hoelfeldt Lund
Jørgen August Bolseth
Thomas Myrseth
Niklas Unneland
Camilla Eriksen
Truls Elverum

Norwegian University of Science and Technology (NTNU)
October 2026

Contents

1 Problem Definition and Business Understanding

1.1 Intro

Aneo is a Nordic renewable energy company headquartered in Trondheim. The company was established in 2022, and is owned by TrønderEnergi and the investment fund HitecVision [**?**]. In addition to producing wind and hydropower, Aneo offers energy market services to other actors in the power sector. Through its Energy Management unit, Aneo helps other companies take part in the power markets in Norway, Sweden and Finland. By using data-driven models for forecasting, optimization and trading, Aneo provides decision support to retailers in the power market, in this case at a fixed price [**?**]. One such service is forecasting electricity consumption for energy retailers, which is the focus of this project. In this case, the forecasts are delivered to a single retailer whose customers are spread across six price areas in Norway and Finland. "The retailer" therefore refers to this one client throughout the report.

1.2 Problem and Decision to be Supported

The problem is to produce a reliable forecast of the retailer's hourly electricity consumption for the next day. That makes a total of 24 values for each of the six price areas. The forecast is issued every day at around 10:00 and can only use information that is available at that time. Because measured consumption data are delivered with a delay of approximately five days, the most recent consumption data available are from five days before the forecast is made (see **??**). The forecast supports the retailer's decision on how much electricity to plan and nominate for each hour and price area. Deviation between the forecast and actual consumption results in an imbalance, which must be settled at prices that can differ substantially from the spot price.

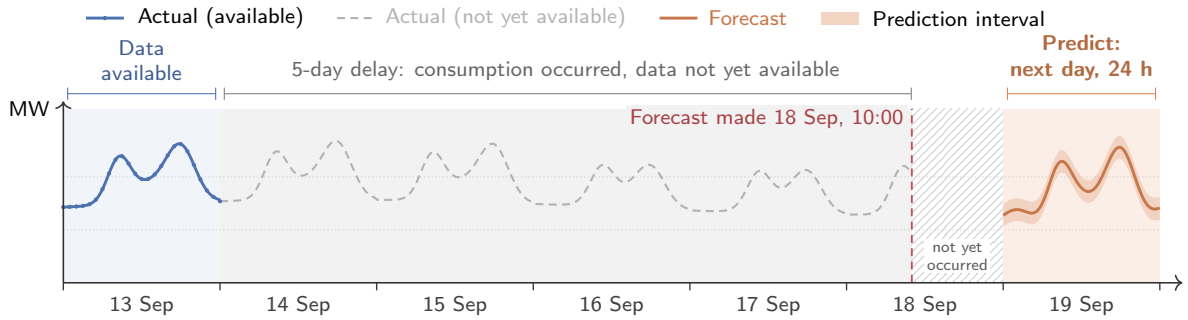


Figure 1: Information available when the forecast is made, illustrated with the example from the task description (synthetic curves). On 18 September at 10:00, measured consumption is only available up to 13 September because of the approximately 5-day data delay, and the remainder of 18 September has not yet occurred. The forecast must cover all 24 hours of 19 September, using historical consumption up to 13 September, the weather forecast for 19 September and calendar information.

1.3 Stakeholders

Aneo Provides next-day hourly consumption forecasts for the retailer's six price areas.

Electricity retailer Buys electricity in the day-ahead market and sells it to its customers with a mark-up.

End customers Buy electricity as needed from their chosen electricity retailer.

Metering data hub (Elhub, Datahub) Delivers metered consumption with a delay of about five days.

TSOs and eSett (Statnett, Fingrid) Keep the system in balance and settle the retailer's imbalances.

1.4 Value Chain and Objectives

The forecasting service connects several actors and stakeholders, as shown in **??**. Weather forecast providers supply Aneo with critical information such as temperature, humidity and wind, while metered consumption reaches Aneo through the metering data hubs with a delay of about five days. Flytt til 1.2

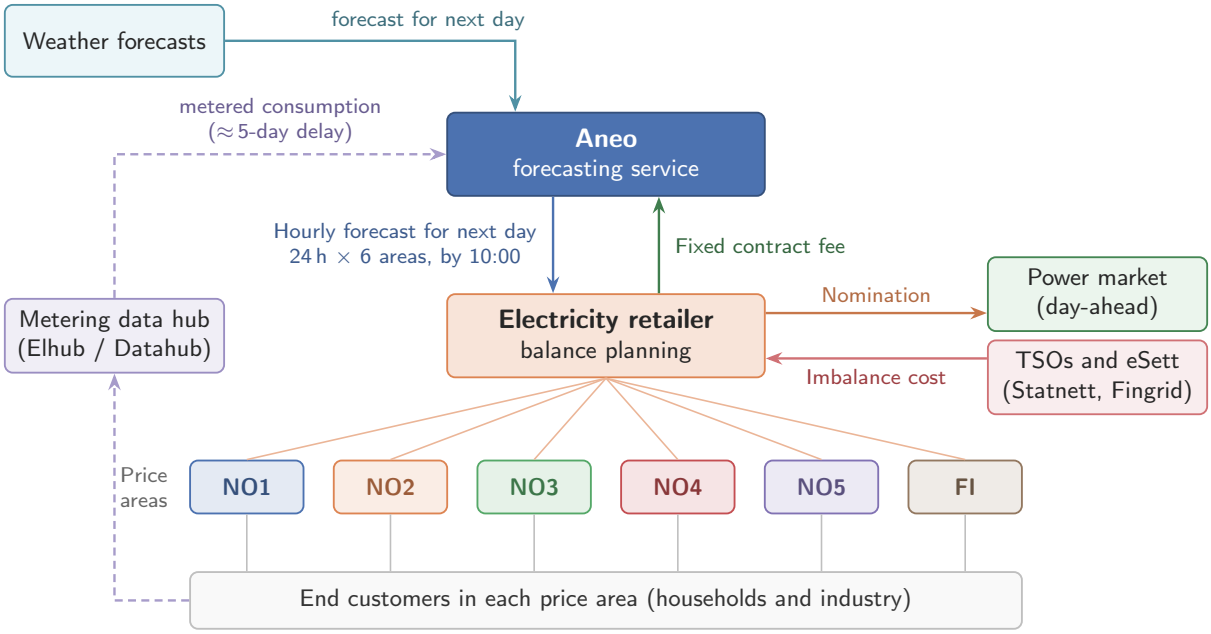


Figure 2: Actors and flows in the case. Aneo delivers next-day consumption forecasts to the retailer, which uses them to nominate demand and bears the imbalance cost.

The quality of the input data is crucial, since a predictive model is only as good as the data it is based on. Errors in the weather forecasts propagate through the whole chain: Aneo's predictions become less accurate, which means that the electricity retailer must settle the resulting deficit or surplus at less favourable imbalance prices, reducing the models value as a decision support tool

Errors can also come from the consumption data, for example missing or delayed meter values, or from changes in the retailer's customer portfolio that the model does not capture. Aneo can also be affected by a poor predictive model: even if the weather forecasts are accurate, the forecasts delivered to the retailer will be wrong if the model uses this information incorrectly. An example could be failing to rank the importance of each source, leading to a wrong combination or formula. Customers are also affected, although only indirectly: if imbalance costs remain high over time, the retailer may pass some of them on through higher prices, which in turn can lead customers to switch to another retailer.

Since the retailer pays Aneo a fixed fee, Aneo does not earn or lose money directly from each imbalance. Instead, the value of the service lies in how it affects the retailer, as illustrated in **??**. More accurate forecasts reduce the retailer's imbalance and financial exposure, which increases the perceived value of the service and the retailer's willingness to pay for it. For Aneo, this ultimately translates into customer satisfaction and a higher likelihood of contract renewal.

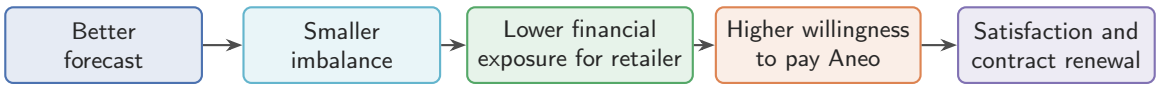


Figure 3: Value chain for Aneo. Because Aneo is paid a fixed fee, forecast quality creates value indirectly, through the retailer's willingness to pay for the service and the likelihood of contract renewal.

1.5 Objectives and Success Criteria

Based on the value chain, the project has three main objectives. Each has a measurable success criterion, evaluated on a held-out test period that the models have not seen during training or tuning (see **??**).

	Objective	Success criterion
Technical	Produce accurate and unbiased hourly forecasts that outperform a simple baseline in all six price areas.	Lower mean absolute error (MAE) than the same-hour-last-week baseline in all six price areas, and a bias close to zero (the choice of MAE is justified in **??**).
Business	Reduce the retailer's total imbalance and the frequency of large forecast errors.	Lower total imbalance (MWh) and fewer hours with large errors than the baseline, consistently across price areas and seasons. Target levels would be agreed with Aneo and the retailer, for example in a service-level agreement.
Operational	Deliver the forecasts reliably before 10:00 every day, using only information available at that time.	The forecast is produced before 10:00 every day, using only data available at that time.

Table 1: Project objectives and the criteria used to evaluate them.

1.6 Constraints and Assumptions

The main constraints are that the forecast is issued at around 10:00 for the next day, that consumption data are only available up to five days earlier, and that only temperature forecasts and calendar information are available as additional inputs (see **??**). We further assume that:

- the retailer pays Aneo a fixed fee, and over- and underforecasting are equally costly;
- imbalance costs scale approximately linearly with the forecast error, so the full settlement mechanism is not modelled;
- the temperature values are forecasts available at the time the consumption forecast is made;
- Aneo has an existing forecasting method, but its details are unknown, so a simple baseline is used as a reference
- the retailer is small relative to each price area, so its forecast errors do not affect the imbalance price;
- the external weather data, which are observed values, are used as an approximation of weather forecasts;
- the retailer nominates directly based on the forecast, without later adjustments in the intraday market.

2 Data Understanding and Background

This section first gives the domain background behind the choice of input variables, and then describes and explores the data.

2.1 Domain Background

Norway is divided into five price areas (NO1–NO5), while Finland is a single price area. Supply and demand within each area, together with trade between areas (limited by transmission capacity), set the market price for each delivery interval in the day-ahead market. Since 1 October 2025 the interval is 15 minutes, but the data and the forecasting task in this project are hourly. The consumption values represent only the retailer's customers in each price area, not the total consumption in the area, so what drives the areas consumption can differ from what drives the area as a whole.

Heating is one of the biggest uses of electricity in cold climates such as Norway and Finland. Households in particular use more electricity in winter, which makes temperature a key input for forecasting demand. Consumption also follows a daily pattern. It rises in the morning when people get up, turn on the lights and use hot water, and in general industry drives demand during working hours. In the evening households push demand up again with cooking and laundry, and at night activity and consumption falls. The hour of the day and the day of the week are therefore important inputs. For this retailer, the dip during working hours in **??** suggests that households make up most of the portfolio.

The retailer submits bids for next-day delivery on Nord Pool, and accepted bids are binding. It bears the financial responsibility for deviations between its purchases and its customers' actual consumption, either directly as a balance responsible party or through an agreement with one. These deviations are settled at the imbalance price, which can differ from the day-ahead price (see **??**), so forecast errors can cause financial losses. The retailer can reduce deviations through intraday trading before delivery, but our analysis ignores this (see **??**).

Public holidays change household routines and business activity and can therefore affect consumption. With only one year of data, the effect is hard to separate from temperature and weekday (see **??**).

2.2 Dataset Description

The dataset `consumption_temp.csv` contains 49,494 hourly observations across five locations in Norway and one in Finland. Each row contains four variables: `time`, the timestamp in UTC; `location`, the location identifier; `consumption`, the average electricity demand during the hour in MW; and `temperature`, the forecast temperature in °C. An average demand of 1 MW over one hour corresponds to an energy consumption of 1 MWh.

The Norwegian locations cover the period from 7 April 2022 at 21:00 to 2 April 2023 at 21:00, with 8,641 observations per location. Finland, labelled `helsingfors` in the dataset, starts on 14 July 2022 at 21:00 and ends at the same time as the Norwegian series, giving 6,289 observations.

Table **??** shows the location labels in the dataset and the price area each represents. The consumption series cover only the retailer's customers in each price area, not total consumption in the area, while the temperature is the forecast for the named city. The consumption level therefore most likely reflects how many customers the retailer has in each area rather than the size of the city, so levels should not be compared directly across areas. The level may also change abruptly if the retailer gains or loses customers, which a model may mistake for a change in demand.

Table 2: Locations, corresponding price areas and observation counts.

Location	Price area	Observations
Oslo	NO1	8,641
Stavanger	NO2	8,641
Trondheim	NO3	8,641
Tromsø	NO4	8,641
Bergen	NO5	8,641
Helsingfors	FI	6,289
Total		49,494

2.3 Exploratory Data Analysis

The exploratory analysis focuses on the patterns that motivate the choice of features and models in **??**.

Consumption level and seasonality regions
The consumption level differs considerably between locations, with Oslo having the highest and Tromsø the lowest across most of the year. Because of these differences in level, forecast errors are also compared relative to the consumption level (nMAE) in **??**. All Norwegian locations show a strong seasonal pattern: in Oslo, the monthly mean rises from about 2.3 MW in the summer of 2022 to about 13.4 MW in December 2022 (**??**).

Not all of this increase is seasonal, however. At similar temperatures, Norwegian consumption in October 2022 is roughly 1.3 to 2 times the level in April 2022; in Oslo, for example, the mean is 2.8 MW at 7.6 °C in April and 6.1 MW at 8.1 °C in October. In Helsinki, consumption increases from about 2.6 to 6.3 MW between July and September 2022. Since temperature alone cannot explain these level shifts, they most likely reflect changes in the retailer's customer portfolio. Lagged consumption features can capture such shifts, while temperature cannot.

Consumption and temperature
Within each location, consumption and temperature are strongly negatively correlated, with correlation coefficients between -0.78 and -0.86 for the Norwegian locations and -0.60 for Helsinki. The relationship is non-linear: consumption is almost flat above about 10–12 °C and increases steeply below, which is consistent with a heating threshold. This makes temperature the most important explanatory variable and motivates models that can capture non-linear effects. **??** shows the relationship per location.

Daily and weekly patterns
The daily profile is clearly shaped by the time of day and by whether it is a weekday or weekend. On weekdays, consumption has a morning peak around 07:00, a dip during the working day and a larger evening peak at around 20:00. On weekends the morning peak is replaced by a later and broader rise from about 09:00, and the evening peak is slightly higher - people like to sleep in. Consumption is lowest at night in all Norwegian locations. Helsinki is almost flat over the day with no clear weekend effect, which suggests that its portfolio differs from the Norwegian ones. These patterns motivate hour-of-day, weekday and weekend features, and show that the hour of day and the day type interact. **??** shows the average profiles.

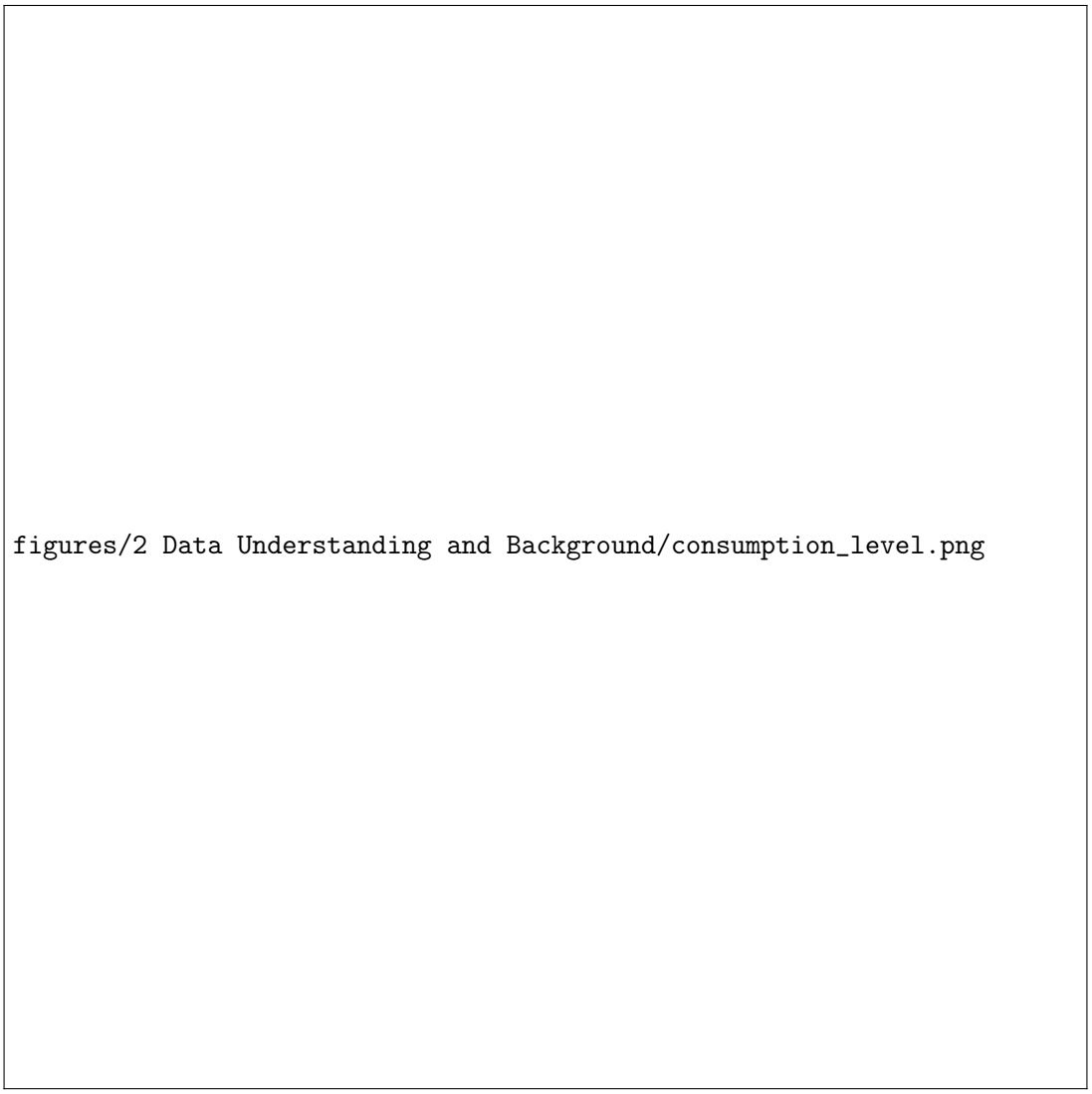


Figure 4: Daily mean consumption per location. The levels differ considerably, and all Norwegian locations rise sharply from October. The Finnish series starts in July 2022 and jumps from about 2.6 to 5 MW within a few days.

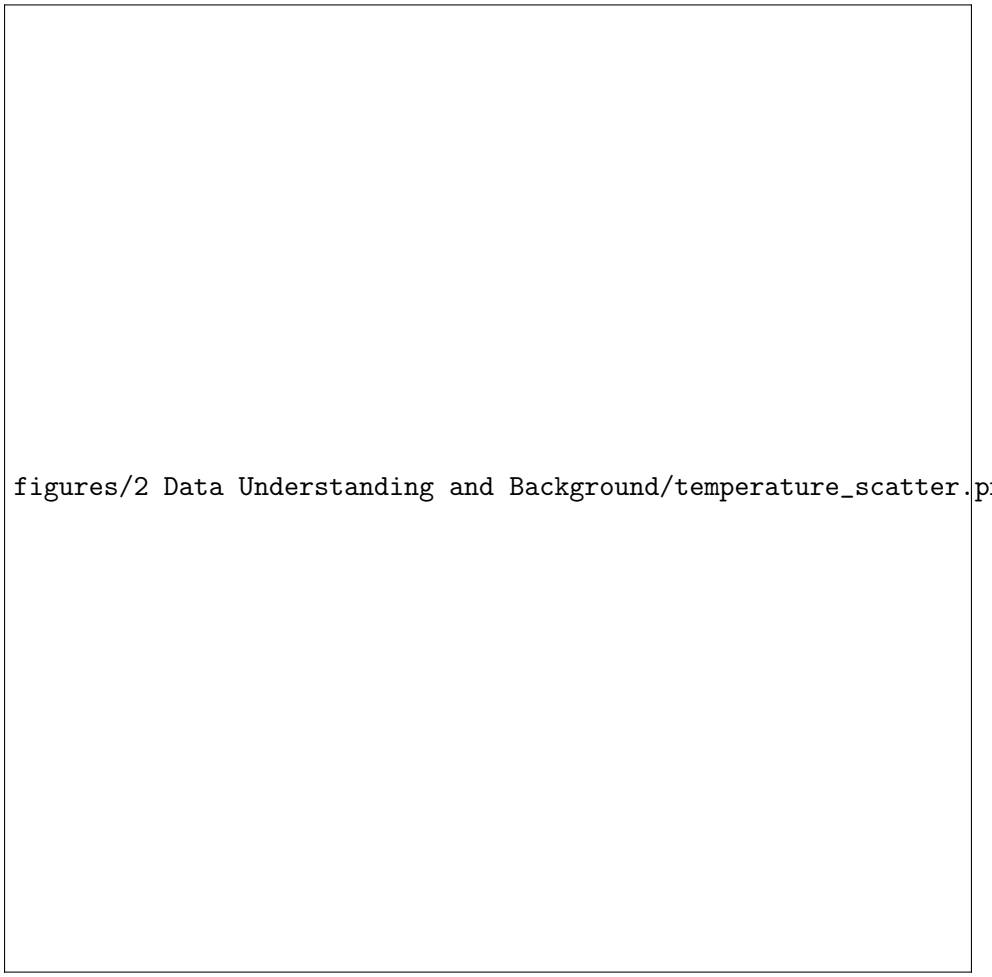


Figure 5: Hourly consumption against temperature forecast per location. Consumption is nearly flat at high temperatures and rises steeply below roughly 10 °C. The separate low-consumption cluster at the same temperatures is the period before the level shift in autumn 2022 (see **??**). Note the different y-axis scales.

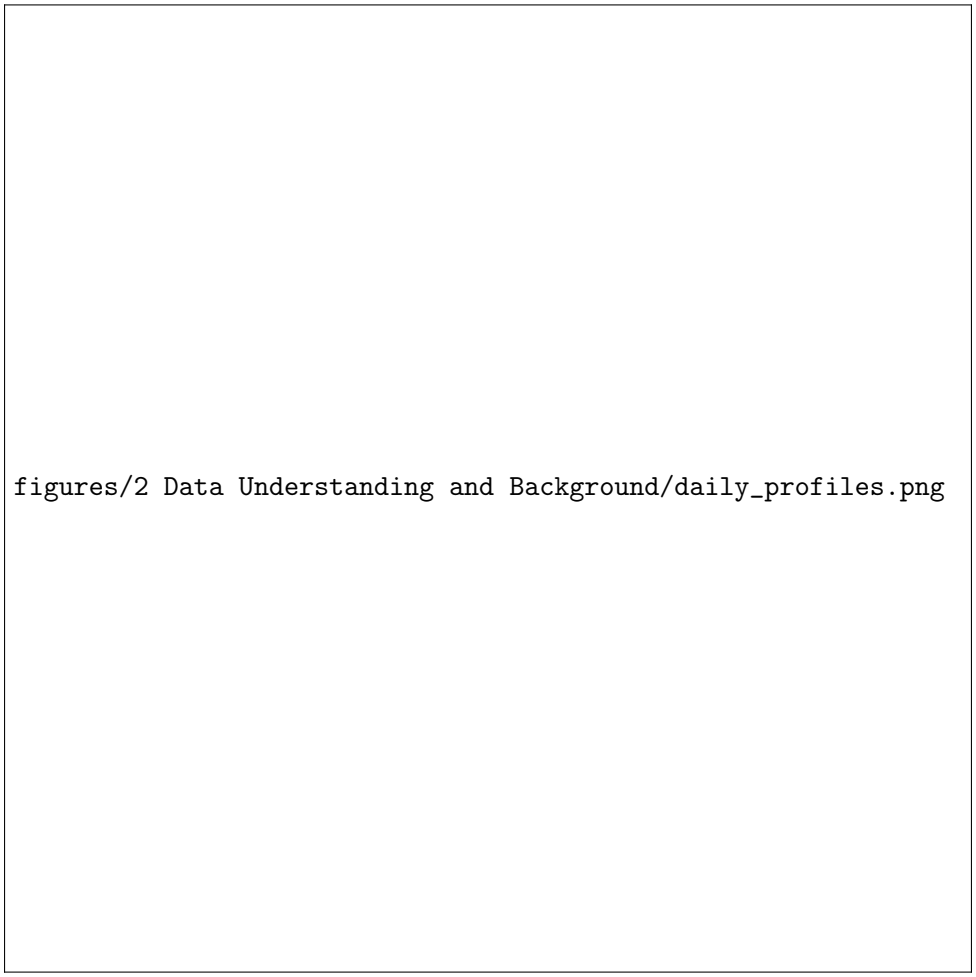


Figure 6: Average daily consumption profile per location, shown relative to each location's mean and in local time, for weekdays and weekends. Norwegian locations have a morning peak on weekdays that is delayed on weekends and an evening peak that is earlier on weekends. Helsingfors is nearly flat.

2.4 Data Quality Issues

An analysis of the data shows that the data is of good quality. There are no missing values, duplicates or gaps, but there are some limitations worth noting. As previously mentioned, the data for Finland start at a later point and have 27.2 % fewer data points, which gives the Finnish price area less training data. The first 32 hours of the Finnish data also have consumption set to 0. While this might seem like an error, it makes sense if the retailer started serving Finnish customers at this point. Other than that the dataset is complete.

The data use UTC, which standardises the timestamps nicely, but Finland and Norway are in different time zones. Norway uses UTC+1 and Finland UTC+2, and both countries use daylight saving time (UTC+2 and UTC+3, respectively). This means that 07:00 in the dataset corresponds to different local times in Norway and Finland, so for training it is necessary to derive local time from calendars. Since timestamps are stored in UTC, daylight-saving transitions introduce no gaps or duplicate timestamps in the dataset. However, converting to local time produces days with 23 or 25 hourly observations, which matters when interpreting daily patterns and constructing calendar features. We derive the local time per location and construct the calendar features from it, as described in **??**.

The temperature values in `consumption_temp.csv` represent forecasts for the corresponding hours rather than observed temperatures. Differences between forecast and actual temperatures introduce uncertainty into the consumption predictions, particularly because temperature influences heating demand. The dataset does not contain matched observed temperatures, so the magnitude of these forecast errors cannot be assessed directly.

2.5 Possible External Data

The original dataset is supplemented with historical weather data, including temperature, wind and precipitation. Weather observations are matched to the consumption series by location and UTC timestamp. Calendar information identifies public holidays separately for Norway and Finland, while time-zone rules provide local dates and hours and account for daylight-saving transitions. The additional weather data describe historical conditions rather than forecasts available before the target hour. Consequently, their use in evaluation may overstate operational forecasting performance, since future wind and precipitation would themselves need to be forecast.

2.6 Assumptions and Limitations

The dataset covers approximately one year, with a shorter observation period for Finland. It therefore contains no repeated annual cycles, and individual annual public holidays are represented at most once. This limits the ability to distinguish recurring seasonal and holiday effects from conditions specific to the observed year.

Weather conditions for each price area are represented by a single city. Temperature, wind and precipitation may vary considerably within an area, so the selected city may not accurately represent the weather experienced by all customers. This introduces uncertainty into the relationship between weather and aggregate consumption.

The dataset contains aggregate consumption without customer-level information. Changes in the retailer's customer portfolio therefore cannot be observed directly. An increase or decrease in consumption could reflect customers joining or leaving the portfolio, changes in existing customers' behaviour, or weather conditions. These explanations cannot be separated reliably using the available data alone.

3 Methods, Analysis and Technical Evaluation

3.1 Reproducing the Forecasting Situation

The information available at forecast time is shown in **??**. We enforce it in three places. Only consumption lag of six days is used as a feature, and the weather forecast and calendar features are taken for the target day itself. Due to the temporal nature of the dataset, the chronological split leaves a gap of six days between the last training day and the first test day, and the same gap is used between the training and validation windows when tuning, so no model learns from consumption that is not yet known when a test day is forecast.

To enforce this, automated tests fail if the target is among the features, or if the gap is too short. A black-box test can also be run on any feature builder: it scrambles every value that is unknown when a given day is forecast, rebuilds the features and checks that the features for that day did not change.

3.2 Data Processing

Prior to training our models, we parse the timestamps as timezone-aware UTC datetimes, which is also the time base used to join the weather data and compute the consumption lag. The calendar features (date, hour, weekday and holiday) are derived from local time, using Europe/Oslo for the Norwegian locations and Europe/Helsinki for Finland, so that for example hour 8 means 08:00 local time in both countries and daylight savings is handled. As mentioned in section 2.4 the dataset is very clean. However, there is still a possibility of missing values at inference time during future deployment, motivating the need for a protocol for handling missing data. Calendar features (hour, weekday, holiday) are deterministic and cannot be missing. The realistic failures are of two kinds, with different causes:

1. the consumption data arrives later than expected meaning we need to predict longer ahead than the model is trained for.
2. the external weather data, which all comes from one provider, might be missing. As it comes from one provider, it will tend to be missing together meaning in some instances we will have no external weather data to rely on.

How correlation motivates the methods for handling missing data

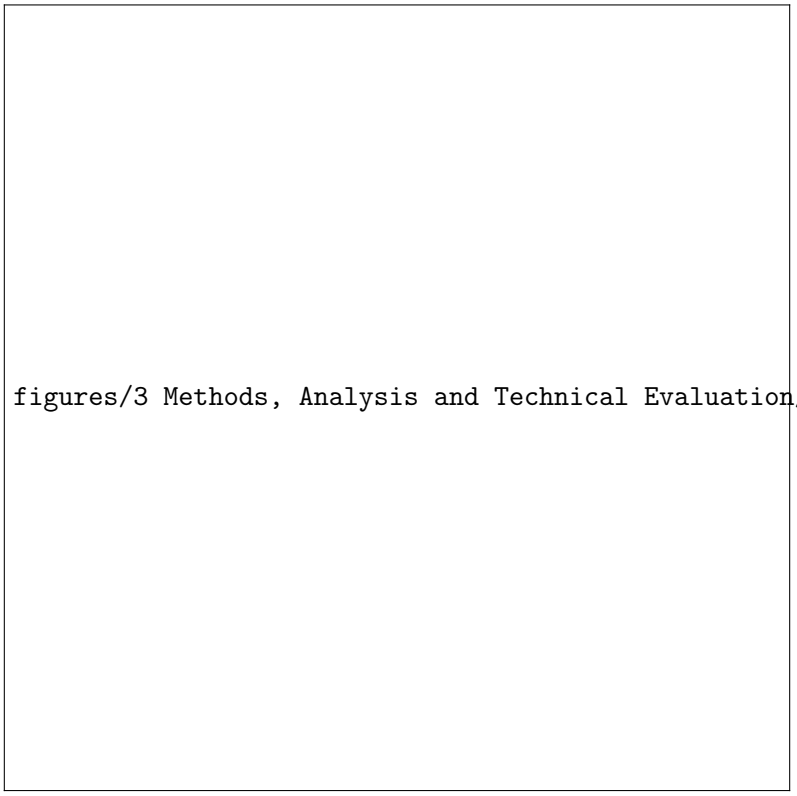


Figure 7: Pearson correlation between the attributes, computed within each location and then averaged over the locations. Computing it within locations avoids the level differences between the cities inflating or masking the correlations.

A value can only be imputed from other features that are correlated with it, so **??** tells us which methods are worth testing. The consumption lags are very strongly correlated with each other ($r = 0.93$–0.98) and with consumption ($r = 0.92$–0.94), so a missing lag can be predicted from the others; this motivates regression-based imputation (MICE, where each feature is regressed on all the others) against simple alternatives such as KNN and mean imputation. The weather variables are only weakly correlated with each other (temperature has $|r| \leq 0.16$ to the rest), so a missing temperature cannot be recovered from them. This motivated the use of interpolation and persistence instead. If all consumption features are missing, the strongest remaining signal is temperature ($r = -0.79$), which motivates a model on weather and calendar only. Pearson correlation only captures linear relationships, and temperature and consumption are related non-linearly, so the correlations guide which methods to test, while the experiment decides which work.

Protocol and experimental set-up

The resulting protocol is applied per row:

1. weather gaps of up to three hours are interpolated, and longer gaps are filled with the value from the same hour the day before, then the training mean;
2. if at most four of the five consumption lag features are missing, they are imputed with MICE (scikit-learn's `IterativeImputer`);
3. if all are missing, the forecast comes from a second model trained on weather and calendar features only.

We tested it on the chronological split used for all models (training 7 April 2022–14 January 2023, 38,235 rows; test 20 January–3 April 2023, 10,495 rows; a six-day gap for the consumption delay). The imputation models are fitted on the training period only, and the forecasting model is XGBoost (500 trees, learning rate 0.05, depth 5, one of the grid-search combinations). Outages are simulated by removing whole days of data for 20 % of the (location, day) pairs in the test period, the same days for all methods, and the MAE is measured on the affected hours.

Results

?? shows the MAE for the four outage scenarios; without any outage it is 0.38 MW on the same hours. The proposed protocol is best or tied among the methods in the table, and no variant in **??** (**??**) is more than 0.03 MW better in any scenario, so we use the simplest. One model per missing-feature pattern would be at most 0.013 MW better but needs several models to maintain.

Table 3: MAE [MW] on the hours affected by a simulated outage (lowest in bold). All methods tried are in **??**.

	Lag 7 d missing	Last day missing	All consumption missing	Weather forecast missing
XGBoost, NaN handled natively	0.70	2.77	3.92	0.62
Mean imputation	0.40	2.56	3.05	0.63
KNN imputation ($k = 5$)	0.39	0.57	2.57	0.69
MICE	**0.38**	**0.40**	2.99	**0.44**
Trained with outages	0.43	**0.40**	1.67	0.57
Proposed: MICE + weather model	**0.38**	**0.40**	**0.89**	**0.44**
Baseline (same hour last week)	2.79	0.57	2.79	0.57

3.3 Feature Engineering

To give the models more information than the raw dataset provides, we engineered additional features in three steps:

1. form a hypothesis for how a feature affects consumption, for example that demand differs on weekends;
2. encode the feature in a form the models can use, for example a binary weekend indicator;
3. test the feature in an ablation study, measuring the change in validation MAE when it is removed.

We ran the ablation study with XGBoost, since it trains quickly enough to refit the model once for each feature removed. To illustrate, a full training job of XGBoost took <20 minutes, while the LSTM took over 6 hours. Each feature was removed one at a time, and the model was refitted and evaluated on the validation windows described in **??**. **??** shows the results.

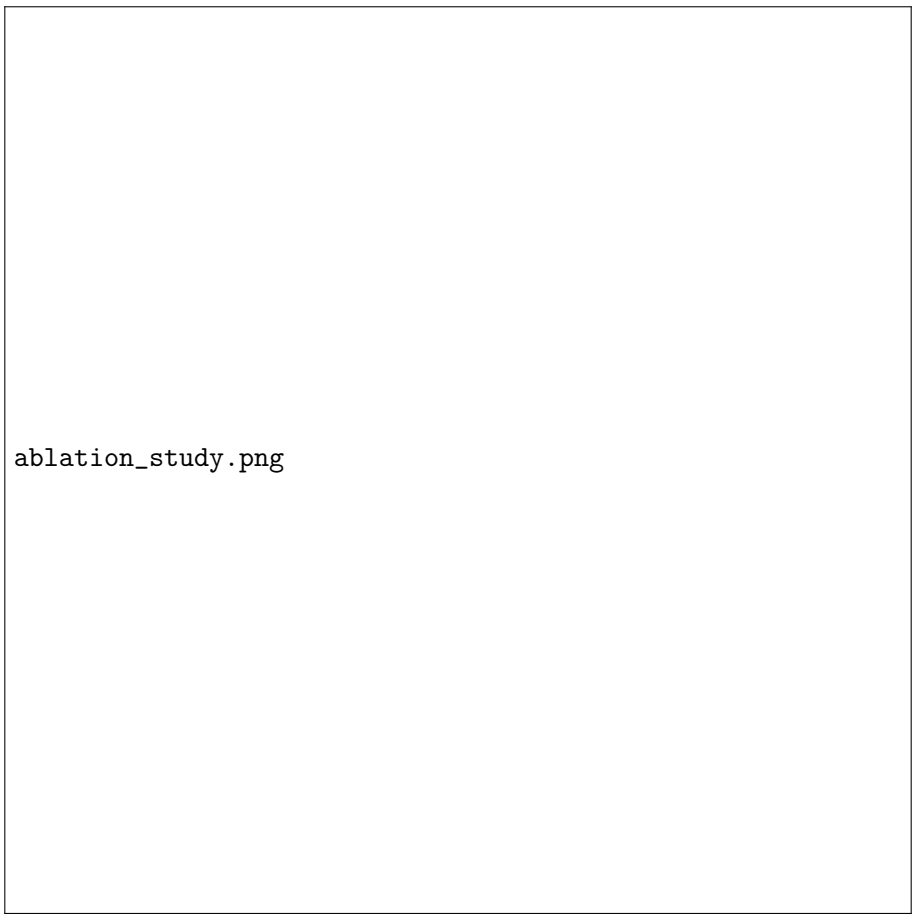


Figure 8: Change in XGBoost validation MAE when each feature is removed one at a time. Features with an effect close to zero, or that improve the model when removed, add little information for XGBoost.

Given the strong correlation between temperature and consumption (**??**), we expected additional weather variables to improve the forecasts. For XGBoost, however, several features had no effect or made the model worse. Wind direction is one example. It is an angle, so 359° and 1° are almost the same direction but far apart as numbers. A decision tree splitting on the raw value cannot see this, and the feature mostly adds noise. The weekend indicator also added nothing, because it is fully determined by the weekday feature: a single split on weekday ≥ 5 (zero-indexed) reproduces it.

Based on these results, we retrained all our main models, a CNN and LSTM discussed in **??**, models without the five features that the ablation flagged: the minimum consumption of the last available day, wind speed, wind direction, snowfall and the weekend indicator. **??** shows the results.

Table 4: Validation MAE before and after removing the five features flagged by the ablation study.

	Validation MAE [MW]		
Model	All features	Reduced set	Δ [%]
XGBoost	0.625	0.591	−5.4
CNN	0.353	0.383	+8.4
LSTM	0.357	0.397	+11.0

The models responded in opposite directions. XGBoost improved by 5.4 % without the five features, while the CNN and LSTM became 8.4 % and 11.0 %

The opposite responses can be explained by how the two model types use weak or noisy features. XGBoost uses features through hard, greedy splits: a noisy feature such as raw wind direction will occasionally be chosen for a split by chance, so the model partly fits noise, and removing it lowers the validation error.

The CNN and LSTM are regularised with weight decay and dropout, which shrinks the weights of uninformative inputs towards zero. For these models a weak or noisy feature costs little, while removing it loses whatever signal it carries. The neural networks also combine all inputs in weighted sums and process them as sequences, so they can learn joint, non-linear, effects, such as wind increasing heating demand on cold days, and how the weather develops through the day.

To conclude we decided on keeping the full feature set in our production model, seeing that the models were able to filter out noise and extract more complex patterns.

3.4 Baseline Model

For the technical aspect of the ML project to pass, we must have a baseline to compare against. As stated in **??**, we assume that there exists an existing forecasting model but that we don have access to it. Thus we need to construct a baseline ourselves to compare our model performance against.

For this project, we were split between using the average consumption for the last seven days and the same consumption as 6 days ago. Our ideal choice would be consumption yesterday, but as we get the consumption data with a 5-day delay, 6 days ago is the closest we can get.

After careful research, we decided on using a *seasonal naïve model* [**?**]. It is defined as follows:

$$\hat{y}_{T+h|T} = y_{T+h-m(k+1)},$$

where m is the seasonal period and k is the integer part of $(h-1)/m$ [**?**]. The prediction for timestep $T+h$ is what happened on the same timestep, one season ago. This works great for periodic time-series data which has a strong repeating pattern. In our case, the data is repeating in a daily oscillating pattern, seen below in **??**. The average performance of the baseline across all cities is an MAE of 0.495 MW, which amounts to 8.98%.



Figure 9: Our data has a clear daily oscillation with a trough at night and a peak during the day. This is a great dataset for a naive seasonal model.

3.5 Main Models – Strengths and Limitations

For consumption forecasting, we evaluated three models: XGBoost, a convolutional neural network (CNN), and a long short-term memory network (LSTM). These were selected to compare gradient-boosted decision trees with neural networks designed to capture temporal patterns. We also explored regional specialization by training separate dense heads for each region on top of a shared CNN.

3.5.1 Gradient-Boosted Decision Trees (XGBoost)

XGBoost was selected due to its strong performance on tabular data. Gradient-boosted decision trees can perform well on relatively small datasets and are computationally efficient to train. The main downside with XGBoost, is as discussed in **??**, it cannot handle smooth transitions as it needs a hard cutoff for its branching.. It does however provide competitive performance without extensive hyperparameter tuning.

Unlike neural networks, XGBoost does not learn hierarchical feature representations through successive neural layers. It also relies on explicitly engineered features, such as lagged consumption values, to capture temporal dependencies. XGBoost therefore serves as a strong machine learning benchmark, allowing us to evaluate whether the additional complexity of neural network architectures translates into meaningful improvements in forecasting performance.

3.5.2 Convolutional Neural Networks (CNN)

We selected a one-dimensional CNN due to its ability to identify local repeating patterns in sequential data through shared convolutional filters. In electricity consumption forecasting, such patterns may include daily increases and decreases in demand. The extracted representations are combined with engineered features in a dense head to produce the final predictions. However, the model's ability to capture longer-term dependencies depends on the length of its input sequences and receptive field.

3.5.3 Long Short-Term Memory Networks (LSTM)

We selected an LSTM to investigate whether modelling sequential dependencies could improve forecasting performance. Unlike a CNN, an LSTM processes observations sequentially and maintains an internal memory state, allowing information from earlier time steps to influence later predictions. This may be useful for capturing gradual changes in consumption patterns. However, LSTMs are more computationally demanding to train and may be prone to overfitting when training data is limited.

3.5.4 Local models per area:

The motivation for training one model per price area is that every price area has a fundamentally different consumption pattern.

However, the global models have over 38,000 rows to train on. Splitting this into per-city training epochs results in <10,000 rows per city. At that level, training complex models without overfitting becomes harder, which means we would have to reduce the number of trees for XGBoost and neurons for the CNN and LSTM. At that point, it is questionable whether the simpler model would be able to have such good predictive power. Likewise, as seen in section 2.3 we see that the cities have a similar response to temperature, signifying that their underlying patterns are similar.

3.6 Validation Approach

Since we are dealing with time-series data, we have to split it chronologically. We decided for an 80–20 split for all models. The training set runs from 2022-04-07 → 2023-01-14, followed by a 6-day gap to avoid data leakage, then the test set running from 2023-01-20 → 2023-04-03.

Both models tune their hyperparameters on the same three consecutive two-week validation windows at the end of the training period: 4–17 December 2022, 18–31 December 2022 and 1–14 January 2023. For each window, the model is fitted on the training data that ends at least six days before the window starts, so the validation mirrors the consumption delay in the train/test split. The test set is never used for tuning, only the validation set. The hyperparameters with the lowest mean MAE over the three windows are then used to refit the model on the full training set, and this model is evaluated once on the test set.

For XGBoost, we ran a grid search (`GridSearchCV`) over the number of trees (100, 500, 1000), the learning rate (0.025, 0.05, 0.075) and the maximum tree depth (5, 10, 15). This gives 27 combinations, each fitted once per validation window. We chose these windows instead of a standard expanding-window split of the whole training period. With only one year of data, the early folds of such a split would be fitted on two to five months of summer data and validated on summer and autumn. This represents neither the test period (January to April) nor the final model, which is fitted on nine months of data. Ultimately, we ended up with 1000 estimators, a learning rate of 0.025 and a max tree depth of 5.

For the neural network, we use Optuna on the same windows. We test different numbers of layers, channels and kernel sizes for the convolution, the width and dropout of the dense head,

and different values for the learning rate, weight decay and batch size of the optimiser. **TODO:** — The neural networks use Optuna to find the optimal hyperparameters over 3 consecutive 2-week periods. We test different numbers of layers, channels and kernels for the convolution, then experiment with the width and dropout in the dense head, as well as various values for the learning rate, weight decay and batch size for the optimisation.

For the LSTM, we test the same values for the dense head, while letting it also optimise for the number of layers and width of its LSTM part.

Seeing as we are validating our hyperparameters on weeks in December and January, there is a danger that our model is better suited for winter than summer, a season that is not tested on.

To counteract this, we limit the number of trials for Optuna to 40, not letting it overfit on cold temperatures.

For the CNN, we end up with one convolution layer, with 16 channels and a kernel size of 5, a width of 32 neurons in the dense head, dropout of 0.24, a 1.5×10^{-3} learning rate, 1.3×10^{-4} weight decay and a batch size of 256.

Meanwhile, the LSTM has a width of 128 in its LSTM part and dense head, dropout of 0.29, a 9×10^{-4} learning rate, weight decay of 5.7×10^{-6} and a batch size of 256.

3.7 Evaluation Metrics

MAE is the main metric:

$$\mathrm{MAE} = \frac{1}{n}\sum_{t=1}^{n} |y_t - \hat{y}_t| \,, \tag{1}$$

where y_t is the actual consumption, $\hat{y}_t$ the forecast and n the number of hours. MAE penalises over- and underforecasting equally and grows linearly with the error. This is in line with how imbalance costs arise for a single retailer: its volume is small relative to each price area, so its own forecast errors do not affect the imbalance price, and the cost grows approximately linearly with the error. To account for the differences in magnitude of regions and hours, we also report nMAE%.

RMSE is reported as well, since large errors tend to occur when the power system is under stress and imbalance prices are high.

Finally, as mentioned in **??**, the bias is a central metric and should be kept close to 0.

3.8 Results

Table 5: Test-set performance per model, all areas pooled ($n = 10\,495$ hours).

Model	MAE [MW]	RMSE [MW]	nMAE [%]	Bias [MW]	Bias [%]
Same hour last week	0.495	0.807	8.98	0.034	0.62
XGBoost	0.334	0.524	6.06	0.153	2.77
CNN (global)	**0.257**	**0.378**	**4.67**	**−0.002**	**−0.03**
LSTM	0.284	0.415	5.15	0.008	0.15
CNN (local heads)	0.295	0.459	5.36	−0.181	−3.29

Consulting **??**, we see all models do quite well, beating the baseline comfortably.

Furthermore, we are pleased to find that the CNN justifies its added complexity and training time by performing the best in all metrics.

One notable result is the large negative bias of the local heads, indicating that they have a tendency to underestimate the consumption. Meanwhile, XGBoost, with its large positive bias, has a tendency to overestimate the actual consumption. The baseline has a low absolute bias, as its total predicted power consumption in MWh is highly similar to the actual power consumption, lagging by one week. Thus, this metric is artificially low for the baseline, meaning that when considering our models, we should use a pinch of salt.

Likewise, I want to discuss why we choose to consider RMSE, while our MAE is the main metric. RMSE is more sensitive to large errors, as the errors get squared. Thus, a high RMSE paired with a low MAE signifies that the model has few but large errors. An RMSE similar to the MAE suggests that all errors are about equal to one another, with few outliers. Looking at our results, we see that the RMSE is higher than the MAE by a similar factor for all our models (RMSE $\approx 1.5\,$MAE). This suggests that the error distribution is the same across all our models. This is positive for the retailer as it means we have zeor to few high imbalance errors.

3.9 Error Analysis and Robustness

To assess the models and locate areas of improvement, we analyse the errors' distribution across price regions, hour of day, weekday and temperature.

Regions: Unsurprisingly, we find a larger MAE for the regions with higher consumption, like Oslo and Stavanger. Due to the differences in magnitude of consumption between the price

areas, we use nMAE% to evaluate. Consulting Table **??**, we find that our models generally have the same level across all areas, except for Helsingfors, where we see a drop in error across all approaches. This improvement is much more pronounced for the baseline, dropping from ∼10% to 2.12%, and XGBoost, ∼7% to 2.47%, while the complex models see more modest improvements.

An interesting finding is that the baseline is the top performer in Helsingfors. This is simultaneously expected and surprising. On one hand, it has less training data, which should mean worse performance, but on the other hand, it has much more stable consumption compared to temperature **TODO:** (reference to figure). We speculate that the straightforward relationship between the temperature and consumption could be part of why the simpler models work so well for Helsingfors.

Hour of day: In Figure **??** (left plot), we find a pattern for the neural networks where errors, reported in nMAE% to stop Oslo from dominating, peak in the morning and afternoon. The morning and afternoon are typically when power usage peaks, which is troublesome, as correcting imbalances in these hours is likely more costly for Aneo's customers.

Weekday: We struggle to find a consistent pattern in errors for the different weekdays (Figure **??**, right plot).

Temperature: In the Nordic countries, power usage naturally peaks when it is the coldest. Since we believe that the cost of an imbalance is higher during peaks, we are interested in whether errors peak too during extreme weather. Dividing the test set into deciles on temperature, we see the nMAE%-curves are flat for XGBoost and the neural networks (Figure **??**). Meanwhile, the baseline struggles with the lowest decile of temperatures, indicating that its inductive bias of assuming weekly repetition struggles when a mild week is followed by a cold week.

3.10 Technical Limitations

With only one year of data available, we face limitations in the training and testing of our models.

Although the models are able to learn general trends, such as the inverse relationship between temperature and consumption, more specific events such as Christmas are past the cutoff point of the training data and thus not trained on. Adding on, we are also relying on our year of data not being an outlier year for the models to generalise well to other years.

On a more positive note, as more data becomes available, we can retrain the models with more features and data points, and possibly see an increase in performance.

Furthermore, the lack of data hampers our ability to assess the models' performance.

First, because the test set is quite small, we experience high statistical variance with only 2 months of data to test on, leaving us vulnerable to outliers.

Second, we can only test our models on a period from late winter to early spring, meaning that our metrics only tell us the models' performance in this period, and not how they will fare in summer and autumn. With a larger dataset, we could test on an entire year and get a better understanding of the models' generalisation.

Another limitation stems from the location column. The cities listed are the largest cities in each region, but the regions are quite large, and we have no information on where in a region Aneo's customers live. For instance, we currently use weather data for Tromsø to predict the consumption of power for consumers in Bodø and Honningsvåg, which are 614 km apart.

4 Business Evaluation, Value and Recommendations

Target length: 3–5 pages (indicative)

4.1 From Forecast Error to Imbalance

TODO: Introduce: since each value is an hourly average in MW, the absolute forecast error for one hour equals the imbalance in MWh. Then define:

As seen, our forecasting models have quite strong predictive power, with a globally defined CNN coming out on top. However, the retailer cares about the total imbalance in MWh (section 1.5), not in the nMAE model performance. Therefore we introduce the following metric, meassuring the imbalance in MWh over areas a and hours t.

$$I = \sum_{a=1}^{6} \sum_{t=1}^{n} |y_{a,t} - \hat{y}_{a,t}| \cdot 1\,\text{h}, \tag{2}$$

$$\Delta I = \frac{I_{\text{baseline}} - I_{\text{model}}}{I_{\text{baseline}}} \times 100\,\%, \tag{3}$$

where I is the total absolute imbalance in MWh, summed over all price areas a and hours t, and ΔI is the percentage reduction in imbalance compared with the baseline. Our final results for every city are given in **??**.

Table 6: Total absolute imbalance and estimated imbalance cost on the test period, summed over all six price areas.

Model	I [MWh]	ΔI [%]	Cost [EUR]	ΔCost [%]
Same hour last week (baseline)	5,192.1	0.0	76,760	0.0
XGBoost	3,504.3	32.5	50,853	33.8
CNN	**2,697.4**	**48.0**	**43,747**	**43.0**
LSTM	2,979.0	42.6	48,137	37.3
Local heads	3,097.5	40.3	47,899	37.6

As we see, the CNN cuts the total imbalance in MWh by almost half, which is of great value for the retailer. However, what is interesting to note is that the cost reduction is not 1:1 correlated with the reduction in imbalance. The key reason for this is that the imbalance market is also affected by the demand side. Thus, if our model has a positive imbalance, and there is an equal positive imbalance on the demand side, the retailer will not have to pay imbalance costs. Likewise, if the imbalances pull in opposite directions, our imbalance costs will grow by more than the imbalance MWh. This phenomenon explains the discrepancy between our imbalance MWh reduction of 48% for the CNN versus an imbalance cost reduction of just 43%. Due to this discrepancy between the reduced imbalance in MWh and in €, we use MWh as our main KPI.

4.2 Business KPIs

TODO: Total absolute imbalance (MWh) per area and in total; % reduction vs. baseline; frequency of large errors (e.g. > X% of consumption); % of hours within acceptable error band; reliability across areas and periods (worst month/area).

As stated in **??**, our main Business KPI is *total imbalance reduction in MWh.* However, it is till interesting to consider total reduced imbalance costs *total hours with an imbalance error over a given threshhold*

By breaking down the total imbalance costs per city, **??**, we get a better idea of the value for the retailer.



Figure 10: Imbalance per city

An interesting finding emerges in the data. Helsingfors is an outlier, where our models did not improve on the baseline. In addition to that, Helsingfors already has the lowest imbalance as a percentage of consumption among the price areas.

This matters a lot for the retailer, as our model is not able to provide value for retailers in Helsingfors. Thus, we fail to meet our business KPIs for this city. For the other cities, our model performance is strong, significantly improving on business KPIs.

Looking at the threshhold, we see that over the two month testing window, spanning all six cities and 10,495 hours, only 15 hours have an imbalance >10% of consumptonm. This is ullustrated in **??**. This is a highly relevant metric for the retailer, as it signifies we have extremely few hours with large imbalance costs.



Figure 11: Error exceedance

4.3 Value for the Retailer and for Aneo

TODO: Retailer: reduced financial exposure and risk. Aneo: higher service value → satisfaction, retention, contract renewal. Discuss how this could be measured (renewal rate, satisfaction surveys, accuracy SLA).

Although Aneo does not directly benefit from good predictions, its customers, the retailer, do. Thus, a strong model which saves money for the retailer through decreasing imbalances, creates value for Aneo through customer satisfaction, and thereby customer retention and possibly new business.

It is therefore clear that the previous KPIs measure direct business value for the retailer, and indirectly for Aneo. Since our client is Aneo, we should provide direct evidence of the value our model creates for them. We can achieve this in a number of ways: by measuring the renewal rate, conducting satisfaction surveys and establishing service-level agreements (SLAs).

Renewal rate is easy to measure, but it is also a lagging signal that can be hard to interpret. If the model performs well, all else being equal, we would expect it to increase or stay the same after deployment. However, in the real world, we seldom have "all else equal". In practice, then, it will be hard to tell whether a decrease in the renewal rate is due to customers being unsatisfied with the service or to other exogenous shocks unrelated to Aneo's service.

Therefore, we should also conduct customer satisfaction surveys. **TODO:** (Jørgen kan dette være noe å monitorere, evt bruke som et tegn på at modell må trenes på nytt?) This helps Aneo learn how the clients experience their service and lets us better understand the renewal rate.

Finally, to decrease the uncertainty regarding the clients' expectations of Aneo, we can establish

service-level agreements (SLAs), the external promise of performance Aneo makes to the retailer. There are several requirements we can imagine a retailer will have of Aneo: timeliness of predictions, average imbalance (nMAE%) per month per area, percent of hours within an acceptable range of ground truth, and improvements in nMAE% over the baseline. The specific numbers for what percentage of predictions should arrive within the 10:00 cutoff, and for what range of hourly imbalance is allowable, are something Aneo should arrive at in consent with individual clients, as they may have different priorities and tolerances.

4.4 Costs, Risks and Trade-offs

TODO: Development and operating cost vs. value; model complexity vs. interpretability/maintenance; risk of large errors in extreme weather; dependence on weather provider and data delivery.

Risks: A key risk for our solution is model drift. Model drift occurs when the model is trained on period a under a specific market regime. As time moves on, the dynamics of the market fundamentally change, resulting in new patterns, which the model is not adapted to. This degrades performance over time, removing business value [**?**].

In our case, model drift occurs when there is a change in the dynamics of the power market. Some key examples include the introduction of Norgespris, which significantly changes consumer behaviour by reducing consumer price sensitivity. Likewise, other infrastructure changes, such as new power cables being connected, like those under the EU ACER agreement, will significantly affect the market dynamics.

Costs: To battle model drift, regular retraining is needed. That is, training the model on newer data so that it can pick up on new patterns. The key issue here is that regular retraining and redeployment come with additional costs from active personnel maintenance in addition to compute costs. Thus, there is a key trade-off to be made between continuously retraining the model to match new data and only training every now and then, when the model performance degrades below a set threshold.

Trade-offs: A key trade-off, in addition to cost–performance, is performance–complexity. A more complex model will have the ability to pick up on weaker and more complex patterns, as demonstarted with the CNNs ability to pick up on patterns from wind, that XGBoost was unable to in **??**, resulting in higher performance.

In a similar manner, there is a trade-off between acquiring more data and model performance. By acquiring more data that is correlated with consumption, we will be able to significantly improve model performance. However, this will come at the cost of licensing data and managing more data streams.

4.5 Recommendations

TODO: Which model to deploy (possibly per area), whether it is good enough to justify continued service, suggested accuracy SLA, what to improve next (more weather variables, shorter data delay, etc.).

TODO: -CNN, but separate solution for helsingfors? -Shorter data delay should be priority -
In the previous chapters, we have proposed 5 approaches and evaluated their analytical performance and business value. We found that the CNN won out in all metrics on the aggregate for the test-set. Moreover, it is the model with the fewest large errors, the lowest total imbalance (exluding baseline) and the one who exposes the retailer to the least amount of financial risk.

Furthermore, we believe the CNN has an acceptable amount of complexity, its training takes around significantly longer than XGBoost, but an order of magnitude shorter than the LSTM. This is an important quality, as we highlighted with the risk of model drift. Thus we expect to retrain the model regurarly as paradigms shift and new data becomes available.

Putting a damper on our solution is all of the models weak performance on Helsingfors (**??**). There we saw the baseline outperform all other approaches, likely due to the stable power consumption reported in the region (**??**).

We therefore recommend proceeding with the CNN for all areas except Helsingfors, where the baseline or existing solution should be used. However, we still recommend setting in an effort for developing a custom model for Helsingfors, specializing on its different consumption pattern.

5 Deployment and Production Readiness

The developed models, and especially the global CNN, forecast the retailer's hourly electricity consumption for the next day with better performance than the same-hour-last-week baseline. However, good test-set results alone do not guarantee that a model is suitable for daily operation, particularly because the performance evidence so far is limited. The models are trained only on about one year of data and tested only on a two-month period covering late winter and early spring. This means that we do not know how they perform in summer and autumn, or in other years. Furthermore, Aneo's existing forecasting method is unknown, so we cannot say whether the CNN improves on what the retailer already receives. We therefore propose a simple system that is tested further and gradually introduced alongside Aneo's existing forecasting method.

5.1 Production Architecture

The forecasting prototype must be turned into a system that delivers 24 hourly values for each of the six price areas by approximately 10:00 every day. Metered consumption arrives with a delay of about five days, and the weather forecast for the target day is available when the forecast is made. Since the retailer needs only one forecast per day, issued at a fixed time, the system does not need to produce predictions on demand or in real time. It only needs to use the latest data available at the time of the run.

We therefore propose a *scheduled batch pipeline*. This is a program that runs at fixed times, collects the data that has arrived since the previous run, computes predictions for all inputs in a single job, and stores the results for delivery [**?**]. In our case, this has three advantages:

1. The pipeline only runs when a forecast is needed, once per day.
2. A failed run can be repeated before the 10:00 deadline.
3. The whole forecast is produced in one run, so it can be checked for completeness and plausible values before it is delivered.

The proposed architecture is shown in Figure **??**.



Figure 12: Proposed production architecture. Validated inputs are turned into features and passed to the forecasting model. The dashed paths show the fallback, which is used when missing inputs cannot be recovered, the model fails, or the output check fails. A scheduler starts the daily run, while versioning and logging support the whole workflow.

Data ingestion and validation. The pipeline needs three inputs: metered consumption from the data hub (Elhub or Datahub), weather forecasts from a provider, and calendar information. Because metered consumption arrives with a delay of about five days, a forecast made on day D for day $D + 1$ may only use consumption up to day $D - 5$. To prevent accidental use of newer and unavailable data, the pipeline stores the time each value was received and uses only data that was available at forecast time. The pipeline also checks the input data for errors before using it, such as external input data that has not arrived on time. If an error cannot be resolved, the fallback described in Section 5.2 is used.

Feature preparation. A preparation step then checks the inputs and turns them into the features the model expects. The same step is used for both training and the daily run, so that the model receives data in the same format in both cases. Calendar features, such as hour of the day and weekday, are calculated in local time for Norway or Finland, as described in Section 3.2. Twice a year, when the clocks change, a local day has 23 or 25 hours instead of 24. On these days the forecast must cover 23 or 25 hours, depending on which way the clock is turned. The delivery format agreed with the retailer should specify how the extra or missing hour is labelled.

Forecasting. The model step loads the forecasting model(s) and produces the forecast. The global CNN is the leading candidate based on the test results in Section 3.8, but the system should not be tied to a single model. In Helsingfors, the same-hour-last-week baseline had lower

error than the CNN (Table 8). The system should therefore allow different models for each price area, making it possible to select the most suitable model based on its performance in each area.

Output validation and delivery. Before delivery, the forecast is checked. Every hour in every price area should have a value, and the values should be within a plausible range. If this check fails, or if missing inputs cannot be recovered, the fallback mechanisms described in Section 5.2 is used. The forecast is then sent to the retailer through a secure interface that the two parties have agreed on. The system also records the delivered forecasts, their delivery times, and the models and data used to generate them. This documents what the retailer received and allows the forecast to be compared with actual consumption once it becomes available.

Beyond this, the system needs a scheduler that starts the daily run, storage for inputs and forecasts, and logs of failures and deliveries. These can start as one simple setup, with the possibility of expanding the infrastructure as the requirements grow.

5.2 Operational Workflow and Fallback Mechanisms

Each day, the scheduler should start the pipeline sufficiently before 10:00 to allow time for input checks, forecasting, delivery, and possible fallbacks. The workflow sequence is: *collect inputs, validate data and construct features, generate forecasts, check outputs, and deliver the results.* The process is complete once the validated forecasts have been successfully delivered to the electricity retailer before the day-ahead gate closure. Each stage should also report any failures, allowing issues to be detected and resolved before they affect forecast delivery.

Input failures. Although the historical dataset contains no missing observations, there is still a possibility of missing values at inference time. To address this, Section 3.2 evaluated a protocol for handling missing data, with promising results. The protocol is therefore a suitable candidate for handling missing values in production. However, since it was tested using XGBoost rather than the better-performing CNN, further testing is needed to confirm that it also works well with the CNN before deployment. If the missing inputs cannot be recovered by the protocol, the system should switch to a pre-approved fallback model that can generate forecasts using the remaining available information.

Model failures. If the forecasting model fails, the system should first attempt to resolve the issue and rerun the forecast before the deadline. If the problem remains, a pre-approved fallback should be used. This could be Aneo's existing forecasting method or the simpler same-hour-last-week baseline. An advantage of the baseline is that it only requires historical consumption data, making it a reliable alternative when external data are unavailable, provided that the required consumption measurements are available. However, since the performance of Aneo's existing solution is unknown, both alternatives should be evaluated before deciding which to use as the fallback.

Forecast validation failures: Before delivery, the system should also check that the forecast contains all required hours and price areas, with no missing, duplicate or unrealistic values. Unusually high predictions should be reviewed rather than automatically rejected, as they may reflect actual peaks in electricity consumption. In this application, delivering a complete and credible forecast on time is at least as important as obtaining a small improvement in forecast performance.

Delivery failures. Even if the forecasting model produces valid forecasts, technical issues may prevent them from reaching the retailer. If delivery fails, the system should automatically retry within the available time before the deadline at approximately 10:00. If repeated attempts fail, an alternative delivery method should be used, such as sending the forecasts through a backup interface. All delivery attempts should be logged, and the system should confirm that the retailer has received the forecasts. This ensures that technical delivery problems do not prevent the retailer from receiving the information needed for its day-ahead planning.

5.3 Testing and Validation Before Deployment

The results in Section 3 show that the models beat the baseline on historical data. However, the test set is very limited. Furthermore, good results on historical data do not show that the production system will work reliably under realistic conditions, or that it is better than what the retailer receives today. We therefore propose three stages of testing and validation prior to deployment, each answering one question: *(1) does the pipeline run correctly*, *(2) are the forecasts as accurate under realistic conditions as the test set suggests*, and *(3) are they better than Aneo's existing forecasts?*

(1) Does the pipeline run correctly? Four tests are needed to check different ways in which the pipeline could go wrong. *Leakage tests* check that the pipeline only uses data available at forecast time. *Replay tests* check that the pipeline reproduces the results of the prototype. *Data tests* check that faulty inputs are caught, and *failure tests* check that a valid forecast is delivered on time even when something goes wrong.

Leakage tests. Data leakage occurs when a model uses information that would not be available at forecast time, potentially making its performance appear better than it actually is. To prevent this, the leakage checks implemented in Section 3.1 should become automatic tests in the production pipeline. These tests scrambles all values that would not yet be known when a day is forecast, rebuilds the features, and checks that the features for that day are unchanged. If they change, the pipeline is using data it should not have. The tests should run whenever the feature code is updated or the model is retrained, ensuring that future changes do not introduce data leakage.

Replay tests. Once the pipeline has been developed, it should be tested by running it day by day over the test period from Section 3. The resulting forecasts from the pipeline should then be compared with those produced by the original prototype. The results should be identical or show only minor numerical differences, which may occur due to differences caused by rounding or software versions. This ensures that converting the prototype into an automated pipeline has not unintentionally changed how data are processed or forecasts are generated, for example through incorrect time-zone handling. Any significant differences should be investigated and resolved before deployment.

Data tests. The pipeline checks its input data for errors before using it (Section 5.1). These checks should be tested with data where errors have been added on purpose, to confirm that they are caught. The tests should also include the two days a year when the clocks change and the day has 23 or 25 hours. These days are not errors, but they are easy to handle wrongly. If the clock change is handled incorrectly, all hours after it are shifted by one hour until the next change. Consumption changes quickly in the morning and evening, so even a one-hour shift can give large errors, and the retailer may nominate the wrong amount for the wrong hours. Because this happens only twice a year, it may not be noticed in normal operation, so it must be tested in advance.

Failure tests. The failures described in Section 5.2 (i.e., input failures, model failures, forecast validation failures and delivery failures) should be simulated, both individually and in combination. Failures are rare, so without simulation the first real test of the fallback would be a day when it is actually needed. A failure test should only pass if the system successfully delivers a valid forecast for the correct date and price area before the deadline, either through the original model or a fallback. Ensuring timely forecast delivery, even when failures occur, is particularly important because the retailer depends on these forecasts to plan its electricity demand nominations. Without a reliable forecast, the retailer has nothing to base its nomination on, which can lead to large imbalances and dissatisfaction with Aneo's service.

(2) Are the forecasts as accurate under realistic conditions as the test set suggests? The models were trained and tested on data from 2022–2023, with the held-out test period

concentrated in winter and early spring. Thus, we do not know how the models will perform in other seasons. The data is also more than three years old, and the electricity market may have changed a lot since then. For example, Norgespris, introduced in 2025, reduces price sensitivity for the consumers it covers, and the retailer's customer portfolio may be very different. Any of these changes could make the results worse than the test set suggests. In addition, some weather variables used in the evaluation, such as rain, are observations and not forecasts that would have been available the day before (Section 2.5). This may overstate the performance of the models. Before the test results can be trusted, Aneo should therefore (a) test the models again on recent data that covers all seasons and the period after Norgespris was introduced, retraining them if needed, and (b) obtain archived weather forecasts issued before 10:00 on each day and repeat the test with them.

(3) Are the forecasts better than Aneo's existing forecasts? Aneo's current forecasting method is unknown, so we cannot yet determine whether the new models perform better. If Aneo has archived its previous forecasts, a comparison can first be conducted using historical data. However, a historical comparison only provides an indication of whether the new models will outperform Aneo's existing method. If the new solution performs worse than Aneo's current method in daily operation, the retailer could face higher imbalance costs and become dissatisfied with the service. Therefore, to gain greater confidence that the new forecasts outperform Aneo's existing forecasts, we also propose running the pipeline in shadow mode. Shadow mode means that the system produces forecasts every day under real operating conditions, but these forecasts are not delivered to the retailer [**?**]. Once actual consumption data become available five days later, the new forecasts can be compared with Aneo's existing forecasts for the same days.

Acceptance criteria. Although the CNN performed best in most of the price areas (Figure **??**), the simpler same-hour-last-week baseline outperformed it in Helsingfors. Therefore, the decision on whether to deploy a new model, and which model to use, should be made separately for each price area. A model should only move on to a pilot in a price area if it meets the following acceptance criteria during the shadow-mode period:

1. *Technical performance:* Lower MAE than both Aneo's existing forecasts and the same-hour-last-week baseline, with a bias close to zero.
2. *Business value:* Lower total imbalance (MWh) and fewer hours with large forecast errors than both Aneo's existing forecasts and the same-hour-last-week baseline, with reduced imbalance costs for the retailer.
3. *Operational reliability:* Forecasts must be generated and ready for delivery before 10:00 every day, using only available data and requiring fallbacks on only a small proportion of days.

The exact thresholds should be agreed upon before the shadow-mode period begins. This prevents the thresholds from being adjusted to fit the observed results. If a model does not meet the acceptance criteria for a particular price area, the existing forecasting method should be retained until the new model has been improved and shown to meet the criteria.

5.4 Reproducibility, Security and Scalability

Reproducibility and versioning. Actual consumption data arrive with a delay of about five days. Therefore, forecast errors may only become apparent five or more days after the forecasts were made. By that time, the code or model may have changed. Aneo should therefore record which code and model versions were used to generate each delivered forecast. This makes it possible to reproduce previous forecasts, investigate unexpected errors and return to an earlier model if an updated version performs worse. In practice, Git can be used to track code changes, while model versions can be stored together with the forecasts they produced.

Security. Although the consumption data are aggregated by price area, they may still contain commercially sensitive information about the retailer. Access to these data should therefore be restricted to authorized personnel, and data transfers between the metering data hubs, Aneo and the retailer should be secure.

Scalability. Since the system only needs to generate forecasts once per day for six price areas, a relatively simple infrastructure should be sufficient. However, the system should be designed so that additional price areas or retailers can be added without major changes. The infrastructure can then be expanded as the requirements grow.

5.5 Deployment Strategy

Even if the models meet the acceptance criteria in Section 5.3, replacing Aneo's existing forecasting solution immediately would introduce unnecessary risk exposure for the retailer. We therefore propose a gradual deployment strategy, where the new solution is introduced in stages and evaluated before expanding its use.

Stage 1 - Shadow mode. The first stage is the shadow-mode described in Section 5.3. During this period, the new solution runs alongside Aneo's existing forecasting method without affecting the forecasts delivered to the retailer. This allows Aneo to evaluate the new models under real operating conditions, fine-tune them based on the results, and determine which price areas meet the acceptance criteria before moving to the next stage.

Stage 2 - Limited pilot. If the shadow-mode results are satisfactory, the new solution should first be introduced in one or two selected price areas. These areas should be chosen based on the improvements demonstrated during testing and the potential financial risk for the retailer. During the pilot, the retailer would use the new forecasts in the selected areas, while Aneo's existing solution continues to serve the remaining areas. The existing method should also remain available as a fallback in case the new solution performs worse than expected. This allows Aneo to evaluate the solution in daily operation while limiting the consequences of potential failures.

Stage 3 - Gradual rollout. If the pilot confirms that the solution is reliable and provides sufficient improvements, it can gradually be introduced in additional price areas. Each area should be evaluated separately, and the existing forecasting method should be retained where the new solution does not provide an improvement. Before expanding, Aneo should also ensure that the necessary monitoring procedures, fallback mechanisms and responsibilities are established, as discussed in Section 6.

Throughout the deployment process, Aneo should be able to return to its existing forecasting method if the new solution fails or no longer meets the agreed performance requirements. This reduces the risk for the retailer while allowing Aneo to benefit from the improvements demonstrated by the new models.

6 Monitoring and Maintenance

Target length: 2–4 pages (indicative)

6.1 Data Monitoring

TODO: Delivery delay of consumption data (does it stay ∼5 days?), missing or late weather forecasts, missing hours, distribution drift in temperature/consumption.

6.2 Forecast Performance Monitoring

TODO: Rolling MAE/nMAE and bias per area (e.g. 7- and 30-day windows) vs. baseline; error by hour; note that actuals arrive with 5-day delay, so monitoring is delayed too.

6.3 Business KPI Monitoring

TODO: Monthly total imbalance (MWh) and estimated cost vs. baseline; reporting to retailer; retailer satisfaction and contract status.

6.4 Failure Modes and Alerts

Table 7: Failure modes, indicators and actions. **TODO:** fill in

Failure mode	Indicator / alert	Action	Owner
Missing weather forecast	No data at 09:30	Use fallback	Aneo ops
Portfolio change (customers gained/lost)	Persistent bias	Adjust level / retrain	Aneo + retailer
Model degradation / drift	Rolling MAE > threshold	Investigate, retrain	Aneo forecast team
Extreme weather	Temp. outside training range	Manual review	Aneo forecast team

6.5 Responsibilities

TODO: Aneo forecast team (model), Aneo ops/IT (pipeline), retailer (data delivery, feedback, informing about portfolio changes), account manager (business KPIs, relationship).

6.6 Maintenance Actions

TODO: Scheduled retraining (e.g. monthly/seasonal) and triggered retraining on drift; yearly holiday calendar update; periodic re-evaluation against baseline; documentation and model changelog.

7 Conclusion

TODO: Summarise the chain: problem → data → analysis → evidence → decision/value → deployment → monitoring. Answer the business question: is the solution reliable and valuable enough? Limitations and future work.

A Additional Figures and Tables

TODO: Per-area results, extra EDA plots, hyperparameters.

A.1 Missing-data experiment

?? lists every method in the missing-data experiment (Section **??**), and **??** the comparison used to set the limit of four missing lags. The experiment is in `machine_learning/missing_data.ipynb`, with the numbers in `machine_learning/results/imputation.json`.

Table 8: MAE [MW] on the hours affected by a simulated outage, all methods (lowest in bold). Without any outage the MAE on the same hours is 0.38 MW.

	Lag 7 d missing	Last available day missing	All consumption features missing	Whole weather forecast missing
XGBoost, NaN handled natively	0.70	2.77	3.92	0.62
Mean imputation	0.40	2.56	3.05	0.63
KNN imputation ($k = 5$)	0.39	0.57	2.57	0.69
MICE	**0.38**	0.40	2.99	**0.44**
MICE with ExtraTrees	**0.38**	**0.37**	2.92	**0.44**
Multiple imputation ($m = 5$)	**0.38**	0.40	2.94	**0.44**
MICE + profile fallback	**0.38**	0.40	2.79	**0.44**
Proposed: MICE + weather model	**0.38**	0.40	**0.89**	**0.44**
Trained with outages	0.43	0.40	1.67	0.57
Trained with outages + indicators	0.43	0.39	1.65	0.60
One model per pattern	**0.38**	0.39	**0.89**	**0.44**
Mean of available lags	0.49	0.50	2.79	0.50
Baseline (same hour last week)	2.79	0.57	2.79	0.57

Table 9: MAE [MW] on outage days when k of the five lag features are removed at random.

k	MICE + model	One model per pattern	Weather and calendar model
1	0.38	0.49	0.89
2	0.39	0.74	0.89
3	0.39	0.84	0.89
4	0.40	0.89	0.89
5	2.99	0.89	0.89

A.2 Error analysis tables and plots

Table 10: MAE [MW] and nMAE% per price area and model.

	MAE					nMAE%				
Area	Same hour last week	XGBoost	CNN	LSTM	Local heads	Same hour last week	XGBoost	CNN	LSTM	Local heads
NO1 Oslo	1.30	0.84	0.58	0.62	0.78	10.67	6.92	4.73	5.04	6.35
NO2 Stavanger	0.59	0.47	0.31	0.37	0.29	10.51	8.24	5.42	6.57	5.12
NO3 Trondheim	0.41	0.23	0.17	0.22	0.20	11.25	6.27	4.65	5.99	5.67
NO4 Tromsø	0.18	0.11	0.10	0.09	0.12	9.41	5.81	5.43	4.54	6.11
NO5 Bergen	0.35	0.20	0.16	0.22	0.19	10.77	6.08	4.99	6.76	6.03
FI Helsingfors	0.14	0.16	0.22	0.19	0.19	2.12	2.47	3.51	3.02	2.95
All	0.49	0.33	0.26	0.28	0.30	8.98	6.06	4.67	5.15	5.36

errorbyhourandweekday.png

Figure 13: Errors by hour of day (left), and weekday (right)

errorsbycold.png

Figure 14: Errors by temperature decile